

Roman ↔ Decimal Converter System

Coursework Report

1. Introduction

This project presents a Python-based application designed to convert numbers between Roman numerals and decimal (integer) representations. The system provides both standard conversion and an optional verbose mode that explains each step of the conversion process. The primary goal of this coursework is to demonstrate the application of Object-Oriented Programming (OOP) principles in a real-world scenario.

Purpose of the application: To provide accurate and user-friendly conversion between numeral systems.

Target users: Students, programmers, and anyone interested in numeral systems.

How to run the program: Execute 'python app/main.py' in a terminal environment.

How to use: Select conversion mode, input a value, and view the result. Additional commands allow saving, loading, and viewing history.

2. System Design and Analysis

The system is modular and follows a layered architecture. Core logic is separated into converters, services, and application layers. This separation improves maintainability, scalability, and readability.

2.1 Functional Requirements

- Convert Roman numerals to decimal numbers.
- Convert decimal numbers to Roman numerals.
- Validate input data before processing.
- Maintain a history of conversions.
- Save and load data from CSV files.
- Provide an interactive CLI interface.

2.2 Object-Oriented Programming Principles

Abstraction: Implemented through the BaseConverter abstract class, which defines a common interface for all converters. This ensures consistency and enforces implementation of required methods.

Encapsulation: Internal data structures such as mappings and history lists are kept private. They are accessed only through methods, preventing unintended modifications.

Inheritance: Specialized converters extend base converters to add functionality such as verbose output without duplicating logic.

Polymorphism: Different converter objects can be used interchangeably through a shared interface, allowing flexible and extensible design.

2.3 Design Pattern: Factory Method

The Factory Method pattern is used to create converter objects dynamically. This approach centralizes object creation and reduces coupling between components. It allows new converter types to be added with minimal changes to existing code.

2.4 Composition and Aggregation

Composition is demonstrated in the `ConversionSession` class, which owns instances of converters and history objects. Aggregation is shown in the `ConversionHistory` class, which can exist independently and be reused across sessions.

2.5 File Handling

The system supports reading from and writing to CSV files. This allows persistent storage of conversion history and demonstrates practical file handling in Python.

3. Implementation Details

The implementation uses Python's object-oriented features to create a clean and extensible system. Each class has a clearly defined responsibility, following the Single Responsibility Principle.

Roman to Decimal Conversion: The algorithm processes characters from right to left, applying addition or subtraction rules based on value comparison.

Decimal to Roman Conversion: Uses a greedy algorithm that subtracts the largest possible Roman value at each step.

Verbose versions extend these implementations to provide step-by-step explanations, improving transparency and learning value.

Error Handling: Input validation ensures that invalid data does not cause runtime errors. Exceptions are used to notify users of incorrect input.

Testing: Unit tests are implemented using the unittest framework. Tests cover conversion accuracy, validation, factory behavior, and session management.

4. Results

- The system successfully performs accurate conversions in both directions.
- Input validation improves reliability and prevents errors.
- The verbose mode enhances understanding of conversion logic.
- Modular design allows easy expansion and maintenance.
- Testing ensures correctness of core functionality.

5. Conclusions

This project demonstrates the effective use of OOP principles and design patterns in Python. The system is modular, extensible, and easy to maintain. It fulfills all functional requirements of the coursework.

Future Improvements:

- Develop a graphical user interface (GUI).
- Extend support for larger number ranges.
- Improve Roman numeral validation rules.
- Add support for additional file formats such as JSON.